

Task 4 - Query Optimization

Before optimization

Execution statistics:

```
{
  "executionSuccess": true,
  "nReturned": 2,
  "executionTimeMillis": 0,
  "totalKeysExamined": 0,
  "totalDocsExamined": 33,
  "executionStages": {
    "stage": "COLLSCAN",
    "filter": {
      "$and": [
        { "category": { "$eq": "Programming" } },
        { "published_year": { "$gte": 2020 } }
      ]
    },
    "nReturned": 2,
    "executionTimeMillisEstimate": 0,
    "works": 34,
    "advanced": 2,
    "needTime": 31,
    "needYield": 0,
    "saveState": 0,
    "restoreState": 0,
    "isEOF": 1,
    "direction": "forward",
    "docsExamined": 33
  }
}
```

How many documents were scanned?

- 33 documents were scanned (indicated by "totalDocsExamined": 33).

Was a collection scan performed?

- Yes. The execution stage is explicitly listed as "stage": "COLLSCAN", meaning MongoDB had to look at every single document in the collection from start to finish.

What was the execution time?

- 0 milliseconds (indicated by "executionTimeMillis": 0). Because my dataset is small (33 documents), MongoDB processed it instantly, but this time will grow dramatically as more data is added.

After optimization

Execution statistics:

```
{
  "executionSuccess": true,
  "nReturned": 2,
  "executionTimeMillis": 1,
  "totalKeysExamined": 2,
  "totalDocsExamined": 2,
  "executionStages": {
    "stage": "FETCH",
    "nReturned": 2,
    "executionTimeMillisEstimate": 0,
    "works": 3,
    "advanced": 2,
    "needTime": 0,
    "needYield": 0,
    "saveState": 0,
    "restoreState": 0,
    "isEOF": 1,
    "docsExamined": 2,
    "alreadyHasObj": 0,
    "inputStage": {
      "stage": "IXSCAN",
      "nReturned": 2,
      "executionTimeMillisEstimate": 0,
      "works": 3,
      "advanced": 2,
      "needTime": 0,
      "needYield": 0,
      "saveState": 0,
      "restoreState": 0,
      "isEOF": 1,
      "keyPattern": {
        "category": 1,
        "published_year": 1
      },
      "indexName": "category_1_published_year_1",
      "isMultiKey": false,
      "multiKeyPaths": {
        "category": [],
        "published_year": []
      },
      "isUnique": false,
      "isSparse": false,
      "isPartial": false,
      "indexVersion": 2,
      "direction": "forward",
      "indexBounds": {
        "category": [
          "[\"Programming\\", \"Programming\"]"
        ],
        "published_year": [
          "[2020, inf.0]"
        ]
      }
    }
  }
}
```

```
    },  
    "keysExamined": 2,  
    "seeks": 1,  
    "dupsTested": 0,  
    "dupsDropped": 0  
  }  
}  
}
```

Documents Examined: Dropped from 33 down to 2 ("totalDocsExamined": 2). MongoDB bypassed 31 irrelevant books entirely because the compound index pointed directly to the 2 matches.

Scan Type: Switched from COLLSCAN to IXSCAN (Index Scan), which feeds straight into a FETCH stage to pull the document data from disk.

Keys Examined: 2 ("totalKeysExamined": 2), meaning it only looked up exactly what it needed in the B-Tree index structure.

Why Indexes Improve Performance

In a database system, data is stored on disk pages in the order it is inserted. As the collection grows, finding specific records becomes increasingly expensive. An index is a specialized, separate data structure (typically a B-Tree) that holds a sorted copy of specific fields alongside a pointer to the actual document's location on disk. Indexes improve performance because they allow the database engine to navigate through a sorted tree structure with a time complexity of $O(\log n)$ rather than scanning every record sequentially with a time complexity of $O(n)$. When a compound index like `{ category: 1, published_year: 1 }` is created, MongoDB organizes the index first by category alphabetically, and then by published_year numerically within each category. This allows the query engine to immediately isolate and jump to the exact subset of documents needed.

Core Differences: COLLSCAN vs. IXSCAN

The two primary execution strategies observed in our testing represent fundamentally different approaches to data retrieval:

- **COLLSCAN (Collection Scan):** During a collection scan, MongoDB must read every single document in the collection sequentially from disk to check if it matches the query criteria. This is highly inefficient because CPU and I/O resources are wasted examining documents that are completely irrelevant to the query (e.g., examining "Fiction" books when looking for "Programming").
- **IXSCAN (Index Scan):** During an index scan, the query engine searches the ordered index structure instead of the full documents. It follows the index bounds to look up the exact keys that match the filter criteria. Once found, the engine uses an internal pointer stage (FETCH) to pull only the matching documents from disk, bypassing all irrelevant data.

Analysis of Documents Examined

- **Before Indexing:** The database engine examined all 33 documents currently residing in the collection just to locate the 2 documents that matched the criteria.

- After Indexing: The engine examined exactly 2 keys in the index and proceeded to inspect exactly 2 documents on disk. It achieved a 100% scanning efficiency ratio ($\text{Documents Examined} \div \text{Documents Returned} = 1$).

Analysis of Execution Time

In our controlled testing environment, the execution time shifted marginally from 0 ms to 1 ms. For small datasets (under 100 documents), a variance of 1 millisecond is standard CPU operating noise and does not represent a degradation. Crucially, as a dataset scales to millions of records, a COLLSCAN execution time will scale linearly ($O(n)$) and quickly lead to multi-second delays and CPU bottlenecks. Conversely, the indexed IXSCAN query will scale logarithmically ($O(\log n)$), maintaining near-instantaneous execution times of a few milliseconds regardless of database growth.